

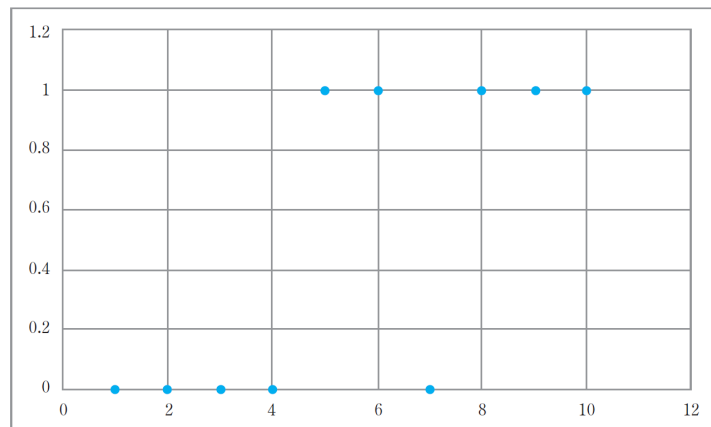
[Chap 10] 로지스틱 회귀



분류에 적합하지 않은 선형 모델

- 일반적인 회귀분석은 연속적(continuous)인 값을 예측
- 반면, 분류 작업은 이산적인(discrete) 값을 예측
 - 예) 와인등급 0, 1, 2, 고객 이탈여부 0, 1
 - 일반적인 회귀분석을 적용하기에 부적합
- 분류 작업의 예: 광고노출횟수에 따른 고객 구매여부

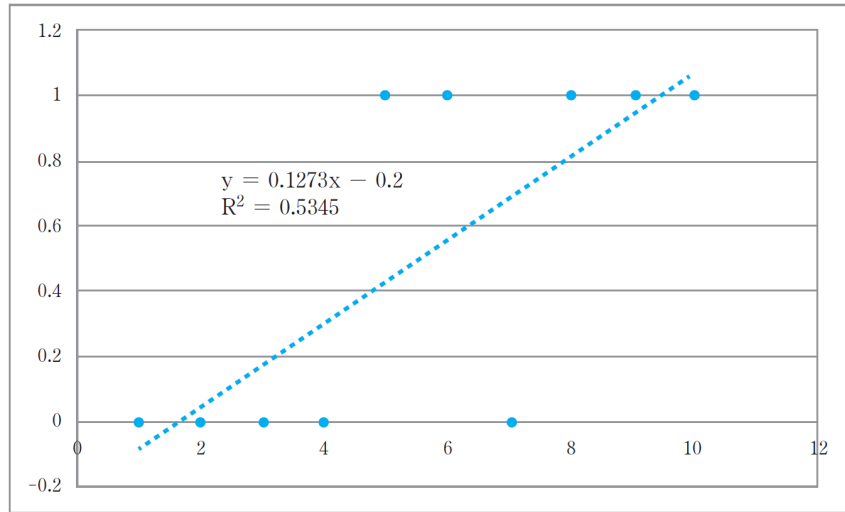
광고노출횟수	구매여부
1	0
2	0
3	0
4	0
5	1
6	1
7	0
8	1
9	1
10	1



[그림 10-1] 광고노출횟수와 구매여부의 산점도

분류에 적합하지 않은 선형 모델 (계속)

- 선형회귀에 의한 구매여부 예측모델은 분류의 목적에 적합하지 않음
 - 광고노출횟수 x 가 0이라면 y 는 -0.2 , 광고노출횟수가 30회라면 y 는 3.62 가 되는 등, $-\infty$ 에서 ∞ 까지 어떤 값이든 나올 수 있게 되어 부적합



[그림 10-2] 광고노출횟수와 구매여부의 선형회귀모형

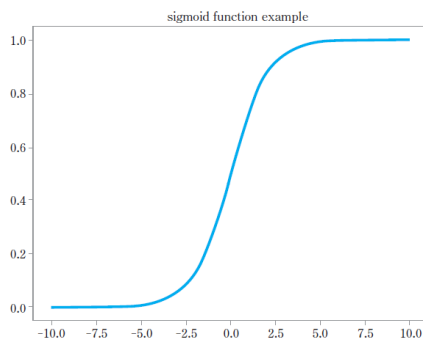
3

시그모이드 함수(sigmoid function)

- S자 형태의 함수를 말하며, 보통 로지스틱 함수(logistic function)와 같은 의미로 사용

$$f(x) = \frac{1}{1 + e^{-x}}$$

- 시그모이드 함수의 시각화



[그림 10-3] 시그모이드 함수의 시각화

예제 ex10-1.py

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def sigmoid(x):
5     y = 1/(1 + np.exp(-x))
6     return y
7
8 x_values = np.arange(-10, 10.1, 0.1)
9 y_values = []
10 for x in x_values:
11     y = sigmoid(x)
12     y_values.append(y)
13
14 plt.plot(x_values, y_values)
15 plt.title('sigmoid function example')
16 plt.show()
```

4

로지스틱 회귀(logistic regression) 모델

- 0 또는 1로 구분하여 예측하기 위한 모델
 - 예) 광고노출횟수에 따른 구매확률 p 를 구하여 $p \leq 0.5$ 이면 1로, 아니면 0으로 판단
- 로지스틱 회귀 모델의 도출
 - 선형회귀 결과식을 $z = wx + b$ 로 표현
 - 공산(odds) : $odds(p) = \frac{p}{1-p}$
 - $0 \leq odds < \infty$ 로서 '그럴직한 정도'를 나타냄
 - 로짓(logit): 공산에 log를 취한 값
 - 임의의 p 에 대해 $-\infty$ 에서 ∞ 사이의 값이 되어, z 의 범위에 대응 가능
 - 선형식의 산출값을 로짓으로 해석
$$\log\left(\frac{p}{1-p}\right) = z$$
$$\frac{p}{1-p} = e^z$$
$$p = \frac{e^z}{e^z + 1} = \frac{1}{1 + e^{-z}}$$
 - 선형회귀식 산출값 z 에 시그모이드 함수를 적용하여 확률로 변환
 - $p \leq 0.5$ 이면 1로, 아니면 0으로 판단: 이진 분류(binary classification)
 - * 여러 가지로 구분: 다항 로지스틱 회귀(multinomial logistic regression)

5

경사하강법을 통한 로지스틱 회귀모델 산출

- 손실함수: 모델의 예측과 실제값의 차이를 나타내는 오차함수
 - 로지스틱 회귀에서는 MSE 대신 엔트로피 함수(entropy function)를 사용
$$L(w, b) = -\frac{1}{n} \sum_{i=1}^n (y_i \log(a_i) + (1 - y_i) \log(1 - a_i))$$
- 예제) 광고노출횟수와 구매여부
 - 고객 i 의 광고노출횟수를 x_i , 구매여부(0 또는 1)를 y_i 로 나타냄
 - 시그모이드 함수의 결과값을 a 로 둠
 - 선형회귀식 : $z_i = wx_i + b$
 - 시그모이드 함수 적용: $a_i = \frac{1}{1 + e^{-z_i}}$
 - 손실함수: $L(w, b) = -\frac{1}{10} \{(y_1 \log(a_1) + (1 - y_1) \log(1 - a_1)) + (y_2 \log(a_2) + (1 - y_2) \log(1 - a_2))$
 $\dots + (y_{10} \log(a_{10}) + (1 - y_{10}) \log(1 - a_{10}))\}$
 - 손실함수는 a_i 들이 y_i 에 가까울수록 0에 근접하게 됨
 - 그래디언트를 통해 손실함수를 줄이는 방향으로 파라미터 수정 가능
 - 손실함수의 그래디언트

$$\frac{\partial L(w, b)}{\partial w} = \frac{1}{10} \sum_{i=1}^{10} (a_i - y_i) x_i \quad \frac{\partial L(w, b)}{\partial b} = \frac{1}{10} \sum_{i=1}^{10} (a_i - y_i)$$

6

경사하강법을 통한 로지스틱 회귀모델 산출 (계속)

예제 데이터에 대해 경사하강법 적용

임의의 초기값

$w = 0.2, b = -0.2$ 으로 두고
그래디언트 계산

- w 와 b 에 대한 그래디언트는
각각 0.3681, 0.1976으로 계산됨

〈표 10-3〉 로지스틱회귀 계산예제 ($w=0.2, b=-0.2$)

i	x_i	y_i	z_i	a_i	$y_i \log(a_i) + (1-y_i) \log(1-a_i)$	$(a_i - y_i)$	$(a_i - y_i)x_i$
1	1	0	0	0.5000	-0.6931	0.5000	0.5000
2	2	0	0.2	0.5498	-0.7981	0.5498	1.0997
...							
10	10	1	1.8	0.8581	-0.1530	-0.1419	-1.4185
손실함수 $L(w, b) = -\frac{1}{10} \sum_{i=1}^{10} (y_i \log(a_i) + (1-y_i) \log(1-a_i))$							0.6147
손실함수의 w 에 대한 그래디언트 $\frac{\partial L(w, b)}{\partial w} = \frac{1}{10} \sum_{i=1}^{10} (a_i - y_i)x_i$							0.3681
손실함수의 b 에 대한 그래디언트 $\frac{\partial L(w, b)}{\partial b} = \frac{1}{10} \sum_{i=1}^{10} (a_i - y_i)$							0.1976

학습률을 0.01로 하고 그래디언트를 사용하여 w 와 b 를 개선

$$w_{new} = w - 0.01 \times \frac{\partial L(w, b)}{\partial w} = 0.2 - 0.01 \times 0.3681 = 0.1963$$

$$b_{new} = b - 0.01 \times \frac{\partial L(w, b)}{\partial b} = -0.2 - 0.01 \times 0.1976 = -0.2020$$

w 와 b 의 개선된 값 0.1963과 -0.2020을 적용하면 손실함수값이 0.6130으로 감소

- 이를 반복하여 더이상 손실함수의 개선이 없을 때까지 계산하면 최종적인 로지스틱 회귀 모델을 얻을 수 있음

7

텐서플로우를 활용한 로지스틱 회귀모델 계산 예제

텐서플로우를 읽어들이고, 확률을 계산하는 함수 구현

```
1 import tensorflow as tf
2
3 def prob(x,w,b):
4     z = w*x+b
5     a = 1/(1+tf.math.exp(-z))
6     return a
```

- (4행) x 와 w, b 를 이용하여 $z = ax + b$ 계산
- (5행) z 를 바탕으로 $a = 1/(1 + e^{-z})$ 계산
- x 가 벡터이므로, a 도 각각의 x 에 대한 계산 결과를 요소로 가진 벡터로 계산됨

손실함수 계산

```
1 def loss(w,b,x,y):
2     a = prob(x,w,b)
3     l = y*tf.math.log(a) + (1-y)*(tf.math.log(1-a))
4     return -tf.reduce_mean(l)
```

- (2행) x, w, b 로 prob 함수를 통해 확률값 a 계산
- (3행) 각각의 a 와 y 의 요소에 대해 $y_i \log(a_i) + (1-y_i) \log(1-a_i)$ 를 계산하여 l 에 대입
- (4행) l 의 각각의 요소에 대한 평균을 계산하고 부호를 -로 하여 엔트로피 함수값 계산

8

텐서플로우를 활용한 로지스틱 회귀모델 계산 (계속)

- 그래디언트 계산

- grad_w 함수는 $(a_i - y_i)x_i$ 의 평균, grad_b 함수는 $(a_i - y_i)$ 의 평균을 계산

```
1 def grad_w(w,b,x,y):
2     a = prob(x,w,b)
3     return tf.reduce_mean((a-y)*x)
4
5 def grad_b(w,b,x,y):
6     a = prob(x,w,b)
7     sum = 0
8     return tf.reduce_mean(a-y)
```

- 데이터값 읽어오기 (데이터 량이 적어서 코드에 직접 기입)

```
1 x = tf.constant([1,2,3,4,5,6,7,8,9,10], dtype=tf.float32)
2 y = tf.constant([0,0,0,0,1,1,0,1,1,1], dtype=tf.float32)
```

- 파라미터 w와 b의 초기값 설정: w=0.2, b=-0.2

```
1 w = tf.Variable(0.2, dtype=tf.float32)
2 b = tf.Variable(-0.2, dtype=tf.float32)
```

- 파라미터 값은 계속 변경해 갈 것이므로 tf.Variable로 설정

9

텐서플로우를 활용한 로지스틱 회귀모델 계산 (계속)

- 학습률을 0.01로 정하고 while 루프로 w와 b를 반복적으로 개선

```
1 learning_rate=0.01
2 while True:
3     h = loss(w,b,x,y)
4     dw = grad_w(w,b,x,y)
5     db = grad_b(w,b,x,y)
6     w.assign_add(-dw*learning_rate)
7     b.assign_add(-db*learning_rate)
8     print(h.numpy(), w.numpy(), b.numpy(), dw.numpy(), db.numpy())
9     if (abs(dw)<0.0001 and abs(db)<0.0001): break
```

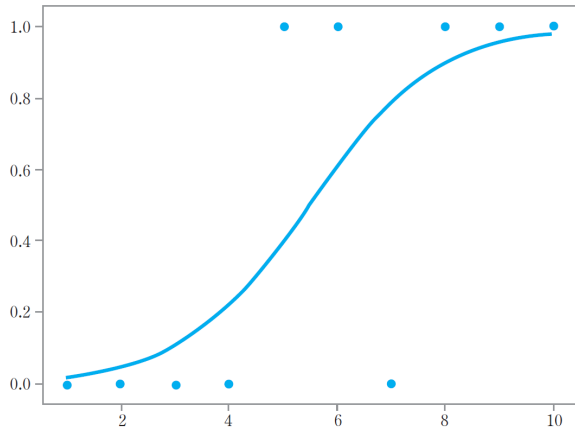
- (4~5행) w와 b의 그래디언트를 구하여 각각 dw, db에 저장
- (6~7행) dw, db에 학습률을 곱한 만큼 w, b를 감소시킴
- (9행) dw와 db의 절대값이 0.0001 미만이면 반복 종료

- 결과 출력 및 모델 시각화

```
1 print("w, b = ", w.numpy(), b.numpy())
2 print("loss = ", loss(w,b,x,y).numpy())
3
4 import matplotlib.pyplot as plt
5 plt.scatter(x, y)
6 plt.plot(x, prob(x,w,b,color='red'))
7 plt.show()
```

실행 결과

- 최종 w , b 는 각각 0.8782, -4.8292로 계산됨
- 확률값 prob 의 그래프가 실제 데이터점들을 잘 반영하여 그려짐을 확인
 - 최종 예측은 확률 0.5를 기준으로 0과 1로 구분하므로, 각각의 x 값에 대한 예측값은 [0,0,0,0,0,1,1,1,1,1]이 됨
 - 정답값은 [0,0,0,0,1,1,0,1,1,1]이므로, 정확도는 80%가 됨



[그림 10-4] 텐서플로우로 계산한 로지스틱 회귀모델 시각화

$w, b = 0.8782071 \ -4.82924$
 $\text{loss} = 0.35656336$

11

사이킷런 패키지를 통한 로지스틱 회귀모델 계산

- 사이킷런 패키지의 로지스틱 회귀 모듈 활용

예제 ex10-3.py

```
1 from sklearn.linear_model import LogisticRegression
2 x = [[1],[2],[3],[4],[5],[6],[7],[8],[9],[10]]
3 y = [0,0,0,0,1,1,0,1,1,1]
4
5 model = LogisticRegression(penalty='none').fit(x,y)
6 print('Predicted = ', model.predict(x))
7
8 w = model.coef_
9 b = model.intercept_
10 print(w, b)
```

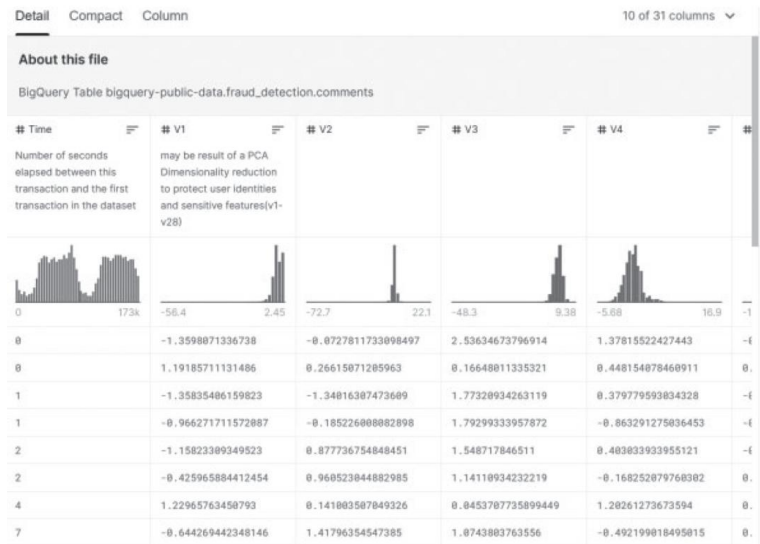
Predicted = [0 0 0 0 0 1 1 1 1 1]
[[0.87969735]] [[-4.83834448]]

- (1행) sklearn.linear_model 모듈의 LogisticRegression 기능을 사용
 - (2행) x 의 값은 각 행별로 각각의 데이터포인트의 값이 들어가야 하므로, 10개 데이터포인트에 대한 각각의 x 값이 별도의 리스트가 되어 각 값이 []로 둘러싸여 있음
 - (5~6행) x 와 y 를 통해 로지스틱 회귀모델 수립 후 예측값 출력
 - (8~10행) 파라미터 w 와 b 의 값 출력 ($w=0.8797$, $b=-4.8383$)
- 앞에서 직접 구현한 결과와 비교하여 예측 결과는 동일, w 와 b 의 값도 비슷함을 확인

12

신용카드 부정사용 예측 모델링 예제

- 로지스틱 회귀로 신용카드 부정사용 예측 모델링
- 데이터는 Kaggle의 공개 데이터 활용
 - Kaggle(<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>)
 - Time: 사용시각
 - Amount: 거래금액
 - 나머지 특성값 열의 이름은 V1, V2... 등의 익명으로 나타남
 - Class: 정답값
 - 0: 정상거래
 - 1: 부정사용



[그림 10-5] 신용카드 부정사용 데이터의 구조

13

신용카드 부정사용 예측 모델링 예제 (계속)

- 데이터를 읽어들이고 내용 확인

```
1 import pandas as pd
2
3 df = pd.read_csv('creditcard.csv')
4 print(df.columns)
5 print('# of rows = ', len(df))
6 print('# of fraud rows = ', len(df[df['Class']==1]))

feature = df[['Time', 'V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7', 'V8',
              'V9', 'V10', 'V11', 'V12', 'V13', 'V14', 'V15', 'V16',
              'V17', 'V18', 'V19', 'V20', 'V21', 'V22', 'V23', 'V24',
              'V25', 'V26', 'Amount']]
target = df['Class']
```

- 총 284807건의 데이터 중 부정사용 492건

- 데이터를 학습용과 테스트용으로 나눔

```
1 from sklearn.model_selection import train_test_split
2 x_train, x_test, y_train, y_test = train_test_split(feature, target, test_size=0.3)
3 print('# of test data = ', len(y_test))
```

- 테스트 데이터로서 전체의 30% 추출
- x_train, x_test, y_train, y_test 변수에 각각 학습데이터의 특성값, 테스트 데이터의 특성값, 학습데이터의 정답값, 테스트데이터의 정답값이 저장

신용카드 부정사용 예측 모델링 예제 (계속)

- 사이킷런의 LogisticRegression으로 모델 수립

```
1 from sklearn.linear_model import LogisticRegression
2 model = LogisticRegression(penalty='none').fit(x_train, y_train)
```

- 테스트 데이터에 대해 예측

```
y_pred = model.predict(x_test)
```

penalty=None, max_iter=1000

- 예측 성능 확인

```
1 from sklearn.metrics import classification_report
2 print("<Classification Report>")
3 print(classification_report(y_test, y_pred))
```

15

신용카드 부정사용 예측 모델링 예제 (계속)

- 실행결과
 - 부정사용 예측인 정답값 1에 대한 예측결과가 중요
 - F1-score 기준으로 73%의 성능을 보임

```
<Classification Report>
              precision    recall  f1-score   support

0               1.00      1.00      1.00     85286
1               0.78      0.69      0.73       157
```

16